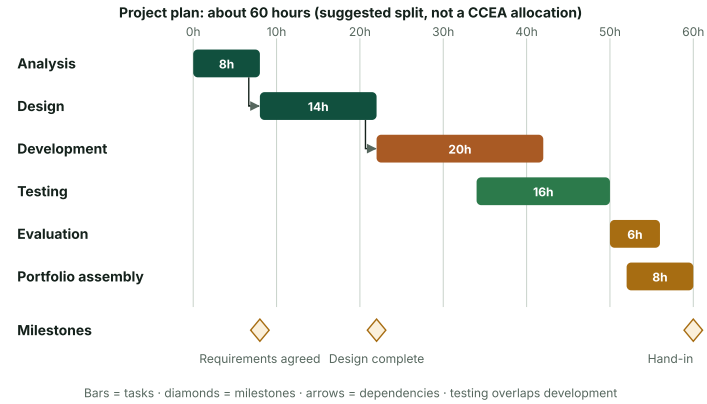
Understanding the task: case study, planning and authentication

AD01 FACT FILE

Specification: students should be able to

- Understand the requirements of the internally assessed A2 2 task: a relational database application produced in response to a CCEA pre-released case study
- Plan and manage a project of about 60 hours using a Gantt chart, milestones and a project log
- Structure the portfolio of evidence so that it follows the stages of the system life cycle
- Understand the rules on academic honesty and authentication of coursework

Key words: case study, client, portfolio, Gantt chart, milestone, dependency, project log, authentication, academic honesty, plagiarism, marking grid, moderation



A Gantt chart for a 60-hour database project showing six tasks, overlapping testing, dependencies and three milestones

The A2 2 task

A2 2 Application Development is the internally assessed unit of the A level, worth 20% of the qualification and taking about 60 hours. Candidates respond to a pre-released CCEA case study by producing a relational database application. Evidence of analysis, design, development, testing and evaluation is presented in a single PDF portfolio. Teachers mark the work against CCEA's marking grid and CCEA moderates a sample. The case study, task instructions and grid should always be checked on the CCEA website for the current year.

Project planning

A Gantt chart lists tasks down the side and time along the top, with a bar for each task's duration. Dependencies link tasks that must happen in sequence, and milestones mark important checkpoints such as agreeing the requirements. The critical path is the longest chain of dependent tasks: any delay on it delays the whole project. A project log records what was actually done, which helps you compare progress with the plan and explain any changes you made to it.

Authentication

Coursework must be the candidate's own work. The candidate and teacher sign declarations, and the teacher must be able to confirm that the work was produced under the required conditions. Sources, tutorials, templates and any permitted AI use must be acknowledged in the portfolio. Copying, collusion or presenting generated content as your own is malpractice and can result in penalties, including disqualification. Keeping dated versions of the database and portfolio shows how the work developed over time.

T Key terms from the mark schemes

Case study
A scenario pre-released by CCEA describing an organisation's problem, which candidates analyse and solve with a database application.

Gantt chart
A planning chart showing each task as a horizontal bar against a timeline, with durations, overlaps and dependencies.

Milestone
A significant checkpoint in a project, such as the completion of a stage, used to monitor progress.

Dependency
A relationship between tasks in which one task cannot start until another has been completed.

Critical path
The longest sequence of dependent tasks in a project; a delay to any task on it delays the whole project.

Authentication
Confirmation by the candidate and teacher that coursework is the candidate's own work, produced under the required conditions.

Exam focus

The moderator only sees your PDF, so evidence must be easy to find and clearly linked. Number your requirements and use those numbers everywhere. Show your process, not just the finished product: a design that changed, with a reason, is stronger evidence than a perfect final screenshot. Avoid unexplained screenshots and generic textbook theory copied into the portfolio; every page should be about your client's system. Read the marking grid headings and make sure each one has clearly labelled evidence.

? Check yourself

1. State what is produced in the A2 2 Application Development task and how the evidence is submitted. [2]
2. Explain why a Gantt chart is useful when planning a 60-hour project. [3]
3. Define the term milestone and give an example from a database project. [2]

Answers: 1. A relational database application that solves the problem in CCEA's pre-released case study (1) / all evidence (analysis, design, development, testing, evaluation) is submitted as a single PDF portfolio (1). 2. A Gantt chart shows each task as a bar against a timeline, so durations and overlaps can be seen (1) / it shows dependencies, i.e. which tasks cannot start until another finishes, e.g. forms need tables first (1) / and milestones, so progress can be checked against the plan and slippage spotted and managed within the 60 hours / hand-in date (1). 3. A milestone is a key point or checkpoint in a project that marks the completion of a significant stage (it has no duration) (1) / example: requirements agreed with the client / design signed off / test plan complete / database tables and relationships built (1).

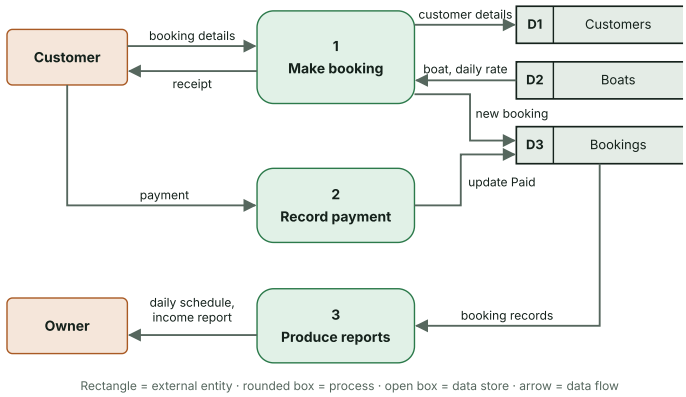
Analysis: client, current system and requirements

AD02 FACT FILE

Specification: students should be able to

- Use appropriate fact-finding methods, including interviewing the client, to investigate a problem
- Describe the current system, its data, processes and problems
- Produce a requirements specification with numbered, measurable functional and non-functional requirements
- Model the proposed system using data flow diagrams

Key words: fact-finding, interview, current system, requirements specification, functional requirement, non-functional requirement, measurable, acceptance criteria, data flow diagram, external entity, data store, context diagram



A level 1 data flow diagram for the proposed Erne Boat Hire system with two external entities, three processes and three data stores

Fact-finding

Analysis begins with fact-finding. Interviews give depth and allow follow-up questions but are time-consuming and reflect one person's view. Questionnaires collect views from many users quickly but cannot be clarified. Observation shows how work really happens, though people may behave differently when watched. Examining existing documents such as booking sheets reveals the actual data items, formats and volumes. In the A2 2 task the case study supplies most facts, and a recorded interview with the client, often role-played by the teacher, adds detail.

Requirements specification

A requirements specification is a numbered list of what the new system must do and how well it must do it. Functional requirements describe data to be stored, processes such as calculations and searches, and outputs such as reports. Non-functional requirements describe qualities such as usability, performance, security and reliability. Each requirement should be specific and measurable, with clear acceptance criteria, so it can be tested and later evaluated as fully, partially or not met. Vague words such as 'easy', 'fast' and 'efficient' need numbers attached.

Data flow diagrams

A data flow diagram (DFD) models how data moves through a system. External entities are sources or destinations of data outside the system. Processes transform data and are numbered. Data stores hold data at rest, and data flows are labelled arrows. A context diagram shows the whole system as a single process with its external entities. A level 1 DFD expands this into the main processes and data stores. Data cannot flow directly between two data stores or between two external entities without a process.

Key terms from the mark schemes

Requirements specification
A numbered, agreed list of the functional and non-functional requirements that the new system must meet.

Functional requirement
A statement of what the system must do, such as the data it stores, the processes it carries out or the outputs it produces.

Non-functional requirement
A statement of how well the system must perform, covering qualities such as usability, performance, security or reliability.

Measurable requirement
A requirement with a specific, testable target so that it can be judged objectively as met or not met.

Data flow diagram
A diagram that models how data moves between external entities, processes and data stores in a system.

Context diagram
A top-level data flow diagram showing the whole system as one process together with its external entities and data flows.

Exam focus

Analysis evidence should be specific to the client in the case study, not general theory. Show the interview questions and answers, then show how each finding became a requirement. Number every requirement and make it testable: if you cannot imagine a test with an expected result, rewrite it. Include non-functional requirements with measurable targets. Make sure DFD labels are meaningful nouns for data flows and verb phrases for processes, and that every flow connects to a process. The same requirement numbers must reappear in testing and evaluation.

? Check yourself

1. Distinguish between a functional and a non-functional requirement, giving an example of each for a boat hire booking system. [4]
2. Explain why the requirement 'The system should be user-friendly' is weak and rewrite it. [3]
3. Give one advantage and one disadvantage of interviewing the client as a fact-finding method. [2]

Answers: 1. A functional requirement states what the system must do – the data, processes and outputs (1) / e.g. the system must calculate the total cost as number of days × daily rate of the boat (1) / A non-functional requirement states how well the system must perform – usability, performance, security, reliability (1) / e.g. only staff with a valid username and password can open the booking forms, or any form reachable from the main menu in no more than two clicks (1). 2. It is vague / subjective – 'user-friendly' means different things to different people (1) / so it cannot be measured or tested objectively, and cannot be evaluated as met or not met (1) / rewritten, e.g.: 'A new member of staff can add a booking in under 2 minutes after a 10-minute demonstration' or 'Every form can be reached from the main menu in no more than two clicks' (1). 3. Advantage, any one: allows follow-up questions to clarify answers / gives detailed, in-depth information about the client's needs (1) / Disadvantage, any one: time-consuming to arrange, carry out and write up / gives only one person's view, which may be biased or incomplete (1).

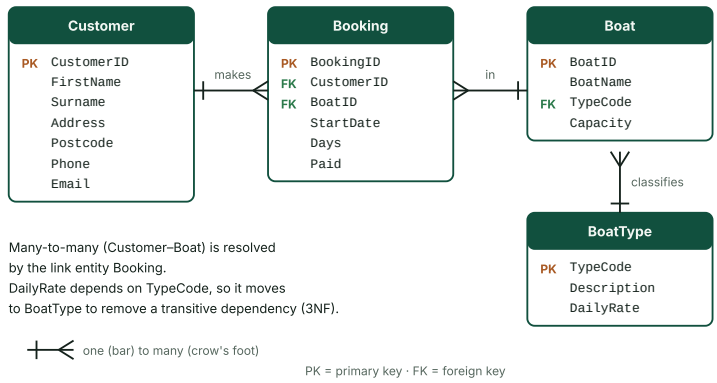
Design: ER diagrams, normalisation, data dictionary and interfaces

AD03 FACT FILE

Specification: students should be able to

- Design a relational database using an entity relationship diagram and normalised tables in third normal form
- Produce a data dictionary showing field names, data types, sizes, validation and keys
- Design user interfaces, reports, queries and algorithms that meet the requirements

Key words: entity, attribute, relationship, ER diagram, primary key, foreign key, normalisation, third normal form, data dictionary, validation rule, wireframe, query design



An ER diagram for Erne Boat Hire in third normal form: Customer and Boat each have a one-to-many relationship with Booking, and BoatType has a one-to-many relationship with Boat

ER diagrams and keys

An entity relationship diagram shows the entities in a system and the relationships between them. Relationships are one-to-one, one-to-many or many-to-many; relational databases cannot implement many-to-many directly, so a link entity is introduced, such as Booking linking Customer and Boat. Each table has a primary key that uniquely identifies each record. A foreign key is a field in one table that is the primary key of another, creating the relationship and allowing referential integrity to be enforced.

Normalisation

Normalisation organises data into tables to remove redundancy and avoid update, insertion and deletion anomalies. First normal form removes repeating groups and makes every field atomic. Second normal form removes partial dependencies, where a non-key attribute depends on only part of a composite key. Third normal form removes transitive dependencies, where a non-key attribute depends on another non-key attribute. A common summary is that every non-key attribute depends on the key, the whole key and nothing but the key.

Data dictionary and interface design

A data dictionary records, for every field, its name, data type, size or format, validation rule, whether it is a primary or foreign key, and a description or example. It ensures consistency and gives the developer exact instructions. Interface design uses annotated wireframes of forms, reports and menus showing layout, controls, navigation, validation and error messages. Designs should suit the users identified in analysis and reference the requirements they meet. Queries and algorithms are designed before they are built.

T Key terms from the mark schemes

Entity relationship diagram
A diagram showing the entities in a system and the one-to-one, one-to-many or many-to-many relationships between them.

Normalisation
The process of organising data into related tables to remove redundancy and prevent update, insertion and deletion anomalies.

Foreign key
A field in one table that is the primary key of another table, used to create a relationship between them.

Third normal form
A table is in 3NF when it is in 2NF and no non-key attribute depends on another non-key attribute.

Data dictionary
A document recording each field's name, data type, size, validation rule, key status and description.

Wireframe
A labelled sketch of the layout of a form, report or screen showing its controls and navigation before it is built.

Exam focus

Show the normalisation process, not just the final tables, and explain each dependency you removed. Make sure your ER diagram, tables and data dictionary agree with each other: inconsistent field names are a common weakness. Every field needs a sensible type and size, and validation must be specific, such as 'between 1 and 14' rather than 'range check'. Annotate wireframes rather than presenting blank sketches. Where you will use macros or VBA code, include pseudocode. Link designs to requirement numbers throughout.

? Check yourself

1. Define the term foreign key. [2]
2. Explain why a many-to-many relationship between Customer and Boat must be resolved, and how it is resolved. [3]
3. A Boat table contains BoatID, BoatName, TypeCode and DailyRate, where the daily rate depends on the type of boat. Explain which normal form is not satisfied and how to correct it. [4]

Answers: 1. A foreign key is a field (attribute) in one table that is the primary key of another table (1) / used to create the relationship between the two tables and allow referential integrity to be enforced, e.g. CustomerID in Booking (1). 2. A customer can hire many boats and a boat can be hired by many customers over time (1) / a relational database cannot implement M:N directly because a field cannot hold a list of keys – fields must be atomic (1) / it is resolved by a link entity, Booking, which holds CustomerID and BoatID as foreign keys, giving two one-to-many relationships: Customer 1–M Booking and Boat 1–M Booking (1). 3. Third normal form is not satisfied (1) / because there is a transitive dependency: DailyRate depends on TypeCode, which is a non-key attribute, rather than directly on the key BoatID (1) / correct it by moving DailyRate into a new table BoatType(TypeCode, Description, DailyRate) (1) / and leaving Boat(BoatID, BoatName, TypeCode*) with TypeCode as a foreign key linking to BoatType (1).

Development: building and evidencing the database application

AD04 FACT FILE

Specification: students should be able to

- Implement a relational database with tables, keys, relationships and referential integrity from the design
- Develop forms with validation, queries including parameter queries and calculated fields, reports and navigation
- Use macros or code to automate processing and improve usability
- Provide annotated evidence of the development of the solution

Key words: table, relationship, referential integrity, validation rule, input mask, lookup, parameter query, calculated field, report, navigation form, macro, annotated screenshot

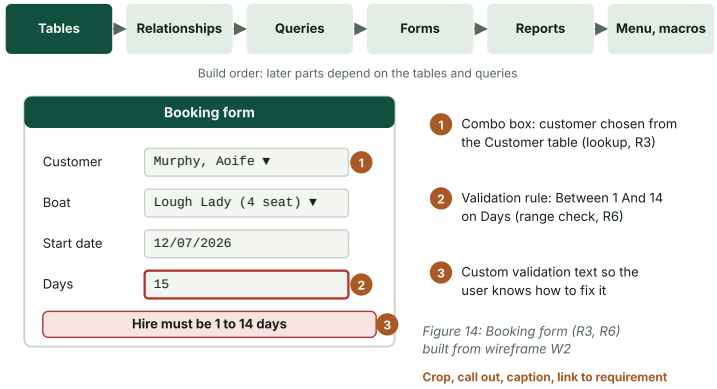


Figure 14: Booking form (R3, R6) built from wireframe W2

Crop, call out, caption, link to requirement

The order for building a database application, and an annotated screenshot of a booking form showing a lookup, a range check and its error message

Implementing the data

Development turns the design into a working application. Tables are created with the data types, sizes, formats and validation from the data dictionary, and each has a primary key. Relationships are created between primary and foreign keys with referential integrity enforced, which prevents orphan records such as a booking for a customer who does not exist. Cascade options must be chosen carefully: cascade delete could remove a customer's entire booking history. Realistic test data should be entered so queries and reports can be checked.

Queries, forms and reports

Queries retrieve and process data. A parameter query prompts the user for criteria when it runs, making one query reusable for any date or surname. A calculated field derives a value from other fields, so derived data such as cost need not be stored. Forms provide a user-friendly way to enter and edit data with validation, combo boxes and clear error messages. Reports format output for printing, often grouped with totals. A navigation menu, macros and VBA code join everything into one easy-to-use application.

Evidence of development

Because the moderator assesses the portfolio rather than the database file, development must be shown through clear, annotated screenshots. Each screenshot should be cropped so it is readable, show the relevant settings (for example design view and the property sheet), and carry a caption with a figure number, the requirement number and the design item it implements. Evidence of iterative development, where a problem was found and fixed, demonstrates understanding and should explain why the change improved the solution.

T Key terms from the mark schemes

Referential integrity
A rule ensuring every foreign key value matches an existing primary key value in the related table, preventing orphan records.

Calculated field
A field in a query, form or report whose value is derived from other fields, such as cost from days and daily rate.

Parameter query
A query that prompts the user to enter a value when it runs and uses that value as its criteria.

Input mask
A pattern that controls the format of data entered into a field, such as a postcode or phone number.

Macro
A stored sequence of actions that automates a task in a database, such as opening a form or printing a report.

Annotated screenshot
A cropped screen capture with labels and a caption explaining what it shows and which requirement and design item it evidences.

C VBA example (Microsoft Access)

Code from a design

```
Private Sub txtDays_AfterUpdate()
    ' R4: calculate the hire cost when the number of days changes
    ' R6: Days must be a whole number from 1 to 14
    Const WEEKLY_DISCOUNT As Double = 0.1
    Dim curRate As Currency
    Dim curCost As Currency

    If IsNull(Me.txtDays) Or Not IsNumeric(Me.txtDays) Then
        MsgBox "Enter the number of days.", vbExclamation, "Booking"
        Exit Sub
    End If
    If Me.txtDays < 1 Or Me.txtDays > 14 Then
        MsgBox "Hire must be 1 to 14 days", vbExclamation, "Booking"
        Me.txtDays.SetFocus
        Exit Sub
    End If

    ' look up the daily rate of the chosen boat
    curRate = Nz(DLookup("DailyRate", "tblBoat", "BoatID = " & Nz(Me.cboBoatID, 0)), 0)
    curCost = curRate * Me.txtDays
    If Me.txtDays >= 7 Then
        curCost = curCost * (1 - WEEKLY_DISCOUNT)   ' weekly discount
    End If
    Me.txtCost = curCost
End Sub
```

Exam focus

Make the sophistication of your solution visible. A complex feature that is not evidenced will not be credited. Show both the setup (design view, SQL, macro or VBA code) and the outcome (the form in use, the query results). Explain why you used a feature, not only what it is. Keep the annotations specific: 'validation rule Between 1 And 14 with custom message meets R6' is far stronger than 'I added validation'. Note and justify any change from your design.

? Check yourself

1. Explain what is meant by referential integrity and why it is important in a booking database. [3]
2. Describe what a parameter query is and give an example from a boat hire system. [3]
3. Explain why the total cost of a booking should be produced by a calculated field rather than stored in the Booking table. [2]

Answers: 1. Referential integrity means every foreign key value must match an existing primary key value in the related table (1) / e.g. a booking cannot be entered for a CustomerID or BoatID that does not exist (no orphan records) (1) / important because bookings must always link to a real customer and boat, so reports, receipts and costs are accurate and the data is consistent; it also prevents a customer with bookings being deleted (if restrict is set) (1). 2. A parameter query prompts the user to enter a value (criterion) when it is run (1) / so one query can be reused for any value instead of creating a separate query for each (1) / example: a query on Booking and Customer with criteria StartDate = [Enter date:] to list all bookings on a date; or search by customer surname (1). 3. The total cost is derived data – it can be calculated from Days × DailyRate whenever needed (1) / storing it would create redundancy and risk inconsistency, e.g. if Days or the daily rate is changed the stored total would be wrong (calculated fields always give the current correct value) (1).

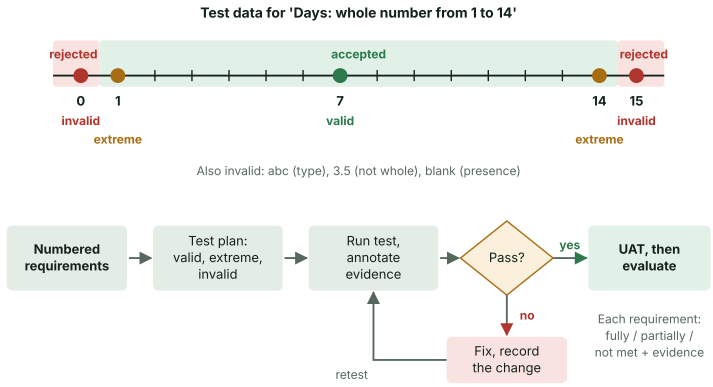
Testing and evaluation against the requirements

AD05 FACT FILE

Specification: students should be able to

- Produce a test plan mapped to the requirements using valid, invalid and extreme (boundary) data
- Carry out testing with annotated evidence of results, corrective action and retesting
- Carry out user acceptance testing
- Evaluate the solution against each requirement as fully, partially or not met, with evidence and justified improvements

Key words: test plan, valid data, invalid data, extreme data, boundary, expected result, actual result, corrective action, retest, user acceptance testing, evaluation, fully met



Choosing valid, extreme and invalid test data for a 1 to 14 range, and the cycle from requirements through testing, fixing and retesting to user acceptance testing and evaluation

Test plans and test data

A test plan lists each test with a number, the requirement it checks, its purpose, the test data, the type of data and the expected result. Columns for the actual result, evidence reference and action taken are completed during testing. Valid data is typical data that should be accepted. Extreme or boundary data lies at the edges of the acceptable range and should also be accepted. Invalid data, including values just outside the range, wrong types and blanks, should be rejected with a helpful message.

Evidence and user acceptance testing

Each test must be evidenced, usually with annotated screenshots showing the data entered and the actual result, cross-referenced by test number. When a test fails, the cause, the correction and a successful retest should all be shown. User acceptance testing involves the client or an end user performing realistic tasks with the finished system, then giving feedback on whether it meets their needs. Their feedback, ideally recorded on a form, provides independent evidence for the evaluation.

Evaluation

The evaluation judges the finished system against every numbered requirement, stating whether each is fully, partially or not met. Each judgement must be supported by evidence, such as test numbers, figures and user feedback, rather than opinion. Partially met and unmet requirements should be explained honestly. Improvements should be realistic, specific and justified, explaining how they would be implemented and how they would benefit the client. Limitations of the solution, such as scalability or remote access, should also be discussed.

T Key terms from the mark schemes

Test plan
A planned list of tests, each with a number, requirement, purpose, test data, data type and expected result.

Extreme data
Data at the boundaries of the acceptable range, which the system should accept; also called boundary data.

User acceptance testing
Testing by the client or end users with realistic tasks to confirm the system meets their requirements.

Valid data
Normal data within the acceptable range that the system should accept.

Invalid data
Data that is outside the acceptable range or of the wrong type, which the system should reject.

Evaluation
A judgement of how well the finished system meets each requirement, supported by evidence, with justified improvements.

Exam focus

A common weakness is testing only validation; test calculations, queries, reports, navigation and non-functional requirements too. Always test both boundaries and the values just outside them. Expected results must be precise: 'error message' is weaker than quoting the message. In the evaluation, avoid descriptive statements such as 'I made a booking form'. Instead write evaluative ones: 'R6 is fully met because tests 1 to 5 show values 1 to 14 are accepted and 0, 15 and text are rejected'.

? Check yourself

1. A field must hold a whole number of days from 1 to 14. Give one example each of valid, extreme and invalid test data. [3]
2. State four columns that should appear in a test plan. [4]
3. Explain why a test plan should be mapped to the numbered requirements. [2]

Answers: 1. Valid (normal): any whole number from 2 to 13, e.g. 5 or 10 (1) / Extreme (boundary): 1 or 14 (1) / Invalid: e.g. 0, 15, -3, 2.5 or 'abc' (1). 2. Any four from: test number (1) / requirement number (1) / purpose / description of test (1) / test data (1) / type of test data (valid, extreme, invalid) (1) / expected result (1) / actual result (1) / evidence reference (e.g. figure number) (1) / action taken / pass or fail (1). 3. So that every requirement is shown to be tested by at least one test and none is missed (1) / and the test results can be used as evidence in the evaluation to judge each requirement as fully, partially or not met (1).

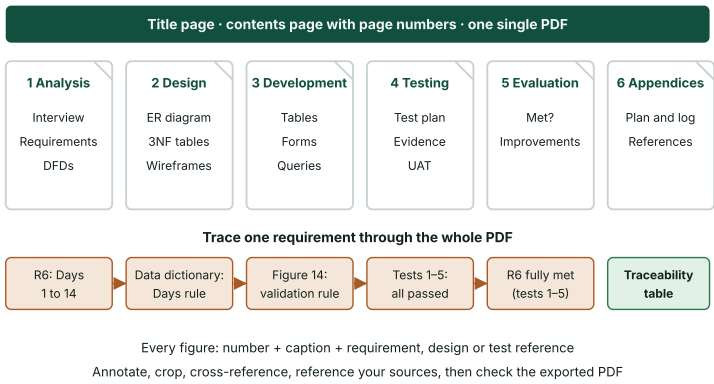
Producing the portfolio

AD06 FACT FILE

Specification: students should be able to

- Present the evidence of analysis, design, development, testing and evaluation as a single, well-organised PDF portfolio
- Annotate evidence clearly and cross-reference it to requirements, designs and tests
- Acknowledge sources and complete the authentication requirements

Key words: portfolio, PDF, contents page, section numbering, figure caption, cross-reference, annotation, appendix, bibliography, referencing, proofreading, checklist



The structure of the single PDF portfolio in six sections, and how one requirement is traced through every section by cross-references

Structuring the PDF

The A2 2 portfolio is submitted as a single PDF, so its organisation directly affects how easily evidence can be credited. A title page, a contents page with accurate page numbers, numbered sections following the system life cycle and consistent heading styles help the reader. Appendices hold supporting material such as the full interview transcript and the project log. Before submitting, export the PDF and check it opens correctly, the pages are in order and the images are clear.

Annotation and cross-referencing

Screenshots are the main evidence of development and testing, so they must be cropped, readable and annotated with callouts that highlight the relevant features. Each figure needs a number and a caption that names the requirement, design item or test it relates to. Cross-referencing these numbers throughout the portfolio allows a moderator to trace any requirement from analysis to evaluation. A traceability table summarising these links is a useful final check that nothing has been missed.

Referencing

All sources that informed the work, including tutorials, websites, templates, images and any permitted use of AI tools, must be acknowledged in a references section using a consistent style. This is part of authenticating the work as the candidate's own. Copying theory or text from websites adds nothing to the evidence and can be treated as plagiarism. Help from the teacher or others should be recorded as the centre requires, and the candidate declaration must be signed.

T Key terms from the mark schemes

Portfolio
The single PDF document containing all the candidate's evidence of analysis, design, development, testing and evaluation.

Traceability table
A table linking each requirement to its design, development, test and evaluation evidence.

Cross-reference
A reference from one part of the portfolio to another, such as a test number or figure number, so evidence can be traced.

Referencing
Acknowledging the sources used in a piece of work in a consistent format, so it is clear which work is the candidate's own.

Exam focus

Presentation does not replace content, but poor organisation can hide good work. Moderators look for clear, relevant evidence matched to the marking grid, so read the current grid and check every heading is covered. Remove padding such as long generic theory or repeated screenshots. Keep explanations concise and specific to your client. Proofread for spelling and technical accuracy, and check that the final PDF matches the version you intended to submit.

? Check yourself

1. State three features that make a PDF portfolio easy for a moderator to navigate. [3]
2. Describe how a screenshot should be presented as evidence of development. [3]
3. What is a traceability table and why is it useful? [2]

Answers: 1. Any three from: a title page (1) / a contents page with accurate page numbers (1) / numbered sections following the stages, e.g. 1 Analysis to 5 Evaluation (1) / PDF bookmarks or a linked contents page (1) / consistent headings, figure numbers and captions (1) / clearly labelled appendices (1) / page numbers on every page (1). 2. Any three from: cropped to the relevant part and readable at normal zoom (1) / annotated with callouts or arrows pointing to the key setting, e.g. a validation rule (1) / a figure number and caption naming the requirement, design item or test number (1) / a sentence or two explaining what it shows and why it was done (1) / show both design view/setup and the result (1). 3. A traceability table lists each requirement with references to where it is designed, developed and tested (and evaluated) (1) / it is a final check that no requirement has been missed and lets the moderator quickly trace and find the evidence for any requirement (1).